



COS6012-B: Concurrent and Distributed Systems

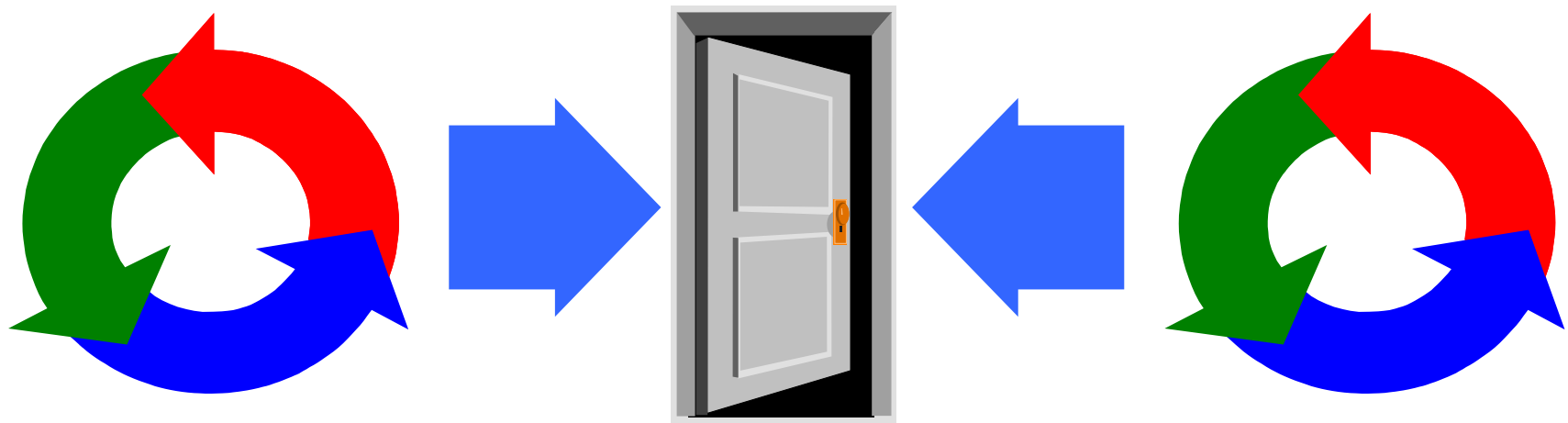
Monitors and Semaphores

Dr Daniele Scrimieri

Outline

- Monitors and condition synchronisation
- Semaphores

Monitors & Condition Synchronisation



Monitors (introduced in previous lab)

- Java relies heavily on a tool called **monitor** which hides some of the details of enforcing mutual exclusion from the user
- So... what is a monitor?
 - *“thread-safe object”*
 - *“synchronization construct that allows threads to have both mutual exclusion and the ability to wait (block) for a certain condition”*
- When writing our code we need to identify *threads* and *monitors*
 - **thread** - **active** entity which initiates actions
 - **monitor** - **passive** entity which responds to actions.

Monitors

- Language features for concurrent programming
- Encapsulate data, which can **only** be observed and modified by monitor's access procedures:
 - **Monitor = encapsulated data + access procedures**
 - Access procedures: mutually exclusive access to data variables
- An object satisfied the data access requirement, since it encapsulates data
 - **private** fields, that can be accessed by object's methods
 - methods can be **synchronized** to provide mutually exclusive access
- So,
 - **Monitor (in Java) = class with synchronized methods**

Monitors and condition synchronisation

- Monitors support condition synchronisation, in addition to ensuring that access to data they encapsulate is mutually exclusive
- **Condition synchronisation** permits a monitor to block threads until a particular condition holds, such as:
 - A counter becoming non-zero
 - A buffer becoming empty
 - New input becoming available
- How is condition synchronisation in monitors modelled and implemented in Java?

Condition synchronisation in Java

Java provides a thread **wait queue** per monitor (actually per object) with the following methods (provided by class **Object** from which all other classes are derived):

- `public final void notify()`
Wakes up a single thread that is waiting on this object's wait set.
- `public final void notifyAll()`
Wakes up all threads that are waiting on this object's wait set.
- `public final void wait() throws InterruptedException`
Waits to be notified by another thread. The waiting thread releases the synchronisation lock associated with the monitor. When notified, the thread must wait to reacquire the monitor before resuming execution.

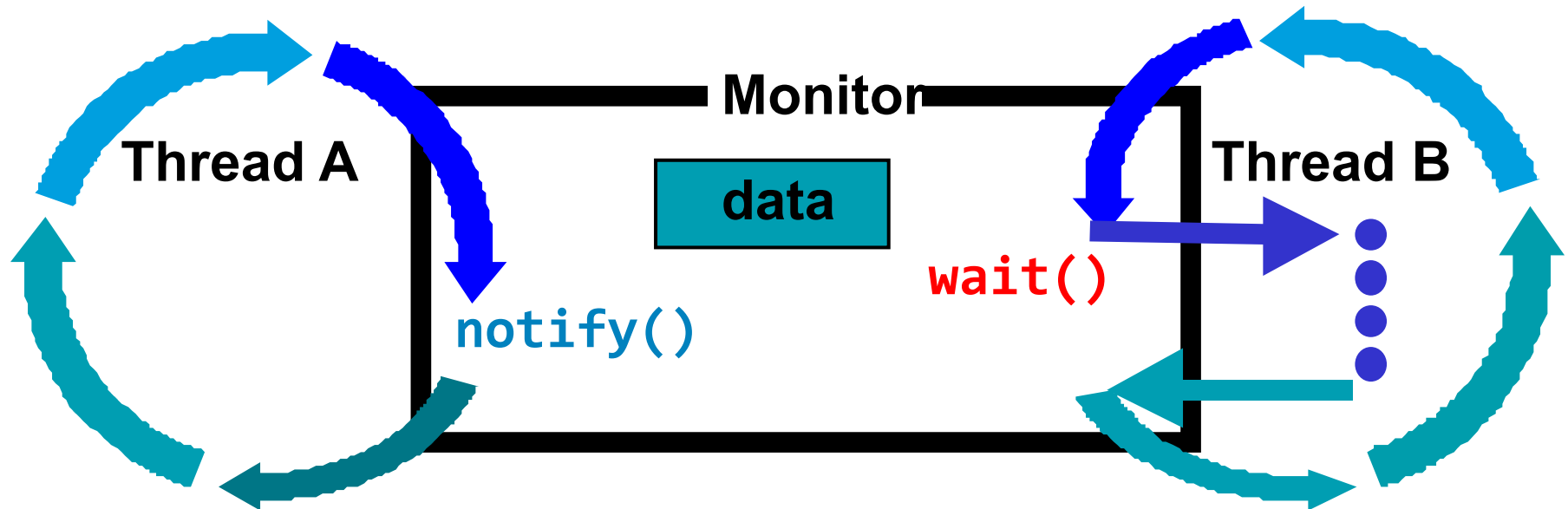
Condition synchronisation in Java

```
public synchronized void myMethod() throws  
InterruptedException {  
    while (!cond)  
        wait();  
    // modify monitor data  
    notifyAll();  
}
```

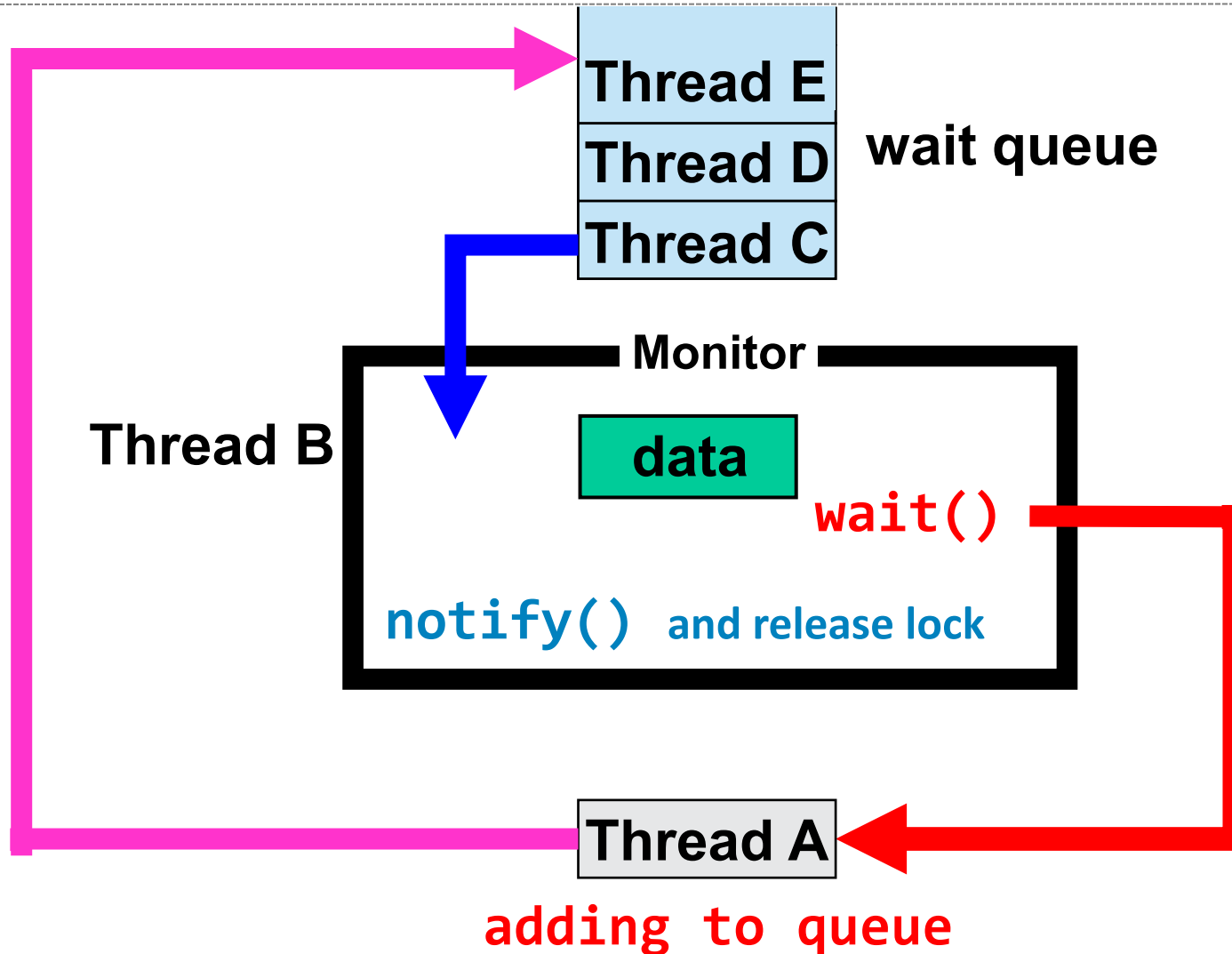
- The while loop is necessary to retest the condition **cond** to ensure that **cond** is indeed satisfied when it re-enters the monitor
- **notifyAll()** is necessary to awaken other thread(s) that may be waiting to enter the monitor, now that the monitor data has been changed

Condition synchronisation in Java

- We refer to a thread **entering** a monitor when it acquires the mutual exclusion lock associated with the monitor and **exiting** the monitor when it releases the lock
- **wait()** - causes the thread to exit the monitor, permitting other threads to enter the monitor



Monitors and condition synchronisation



Monitors - More general definition

Monitors have four components:

- a set of **private variables** which represent the state of the resource
- a set of **monitor procedures** which provide the public interface to the resource
- a set of **condition variables** used to implement condition synchronisation
- **initialisation** code which initialises the private variables

Monitor procedures

Monitor procedures manipulate the values of the private monitor variables:

- only the names of monitor procedures are visible outside the monitor
- the only way a process can read or change the value of a private monitor variable is by calling a monitor procedure
 - the private monitor variables are shared by all the monitor procedures
 - monitor procedures may also have their own local variables – each procedure call gets its own copy of these
- statements within monitor procedures (or initialisation code) may not access variables declared outside the monitor (unless passed as arguments to a monitor procedure)

Condition variables

Condition variables are used to delay a process that can't safely execute a monitor procedure until the monitor's state satisfies some Boolean condition:

- condition variables are not visible outside the monitor and the only access to them is via **special monitor operations** within monitor procedures
- their values can't be tested or assigned to directly even by the monitor procedures!

Synchronisation

Synchronisation within monitors is achieved using monitor procedures and condition variables:

- **mutual exclusion** is **implicit** - monitor procedures by definition execute with mutual exclusion
- **condition synchronisation** must be **programmed explicitly using condition variables**

Mutual exclusion

At most **one** instance of **one** monitor procedure may be active in a monitor at a time:

- if one process is executing a monitor procedure and another process calls a procedure of the same monitor, the second process blocks and is placed on the **entry queue** for the monitor
- when the process in the monitor completes its monitor procedure call, mutual exclusion is passed to a blocked process on the entry queue
- entry queues are usually defined to be **FIFO**, so the first process to block will be the next one to enter the monitor

Critical sections

- This solves the **critical section problem**:
 - if all the shared state in the system is held in private monitor variables and
 - all communication between processes is via calls to monitor procedures; then
 - any accesses to any part of the shared state by any process is **guaranteed to be mutually exclusive** of any other accesses to that part of the state by other processes.
- Different critical sections can be implemented using a different monitor for each critical section

Condition synchronisation

The value of a condition variable is a **delay queue** of blocked processes waiting on a condition:

- if a call to a monitor procedure can't proceed until the monitor's state satisfies some boolean condition, the process that called the monitor procedure **waits** on the corresponding condition variable
- when another process executes a monitor procedure that makes the condition true, it **signals** to the process(es) waiting on the condition variable

Operations on condition variables

We assume that the following operations are defined for a condition variable v :

- **`wait(v)`** wait at the end of the delay queue for v
- **`signal(v)`** wake the process at the front of the delay queue for v and continue
- **`signal_all(v)`** wake all the processes on the delay queue for v and continue
- **`empty(v)`** true if the delay queue for v is empty

The wait operation

- If a process can't proceed, it blocks on a condition variable **v** by executing:

wait(v);

- The blocked process relinquishes exclusive access to the monitor and is appended to the end of the delay queue for **v**

The `signal` operation

- Processes blocked on a condition variable **v** are woken up when some other process performs a signal operation on the variable:

`signal(v);`

- This awakens the process at the front of the delay queue. If the delay queue for **v** is empty, signal does nothing.

Signalling disciplines

- When a monitor procedure calls **signal** on a condition variable, it wakes up the first blocked process in the delay queue waiting on the condition
- What does the signaller process do after calling signal?
 - **Signal and Wait**: the signaller waits until some later time and the signalled process executes now
 - **Signal and Continue**: the signaller continues and the signalled process executes at some later time
- The following examples use the **signal and continue** signalling discipline

Summary: Monitors & Java

Monitors form the basis of Java's support for (shared memory) concurrency:

- **Mutual exclusion** can be implemented in Java using the **synchronized** keyword
- A synchronised method (or block) is executed under mutual exclusion with all other synchronised methods on the same object
- Java provides basic operations for condition synchronisation: **wait()**, **notify()**, **notifyAll()**
- Each Java object has a single (implicit) condition variable and delay queue, the **wait set**
- Other operations are provided for condition variables
- Java uses the **signal and continue** signalling discipline

Monitor example in Java: Carpark

- A controller is required for a carpark, which only permits cars to enter when the carpark is not full and permits cars to leave when it is not empty
- Car arrival and departure are simulated by separate threads
- Events or actions of interest?

arrive and **depart**

- Guarded actions!
- We need to identify threads and monitors
 - thread - active entity which initiates actions
 - monitor - passive entity which responds to actions

CarParkControl monitor

```
class CarParkControl {  
    private int spaces;  
    private int capacity;  
  
    CarParkControl(int n)  
        {capacity = spaces = n;}  
  
    synchronized void arrive() {  
        [...]  
        --spaces;  
        [...]  
    }  
  
    synchronized void depart() {  
        [...]  
        ++spaces;  
        [...]  
    }  
}
```

Mutual exclusion by
synch methods

Condition
synchronisation?

Block if full?

Block if empty?

Signalling?

CarParkControl monitor

```
synchronized void arrive() throws  
InterruptedException {  
    while (spaces==0) wait();  
    --spaces;  
    notifyAll();  
}
```

Block if full

notify() or notifyAll()?

```
synchronized void depart() throws  
InterruptedException {  
    while (spaces==capacity) wait();  
    ++spaces;  
    notifyAll();  
}
```

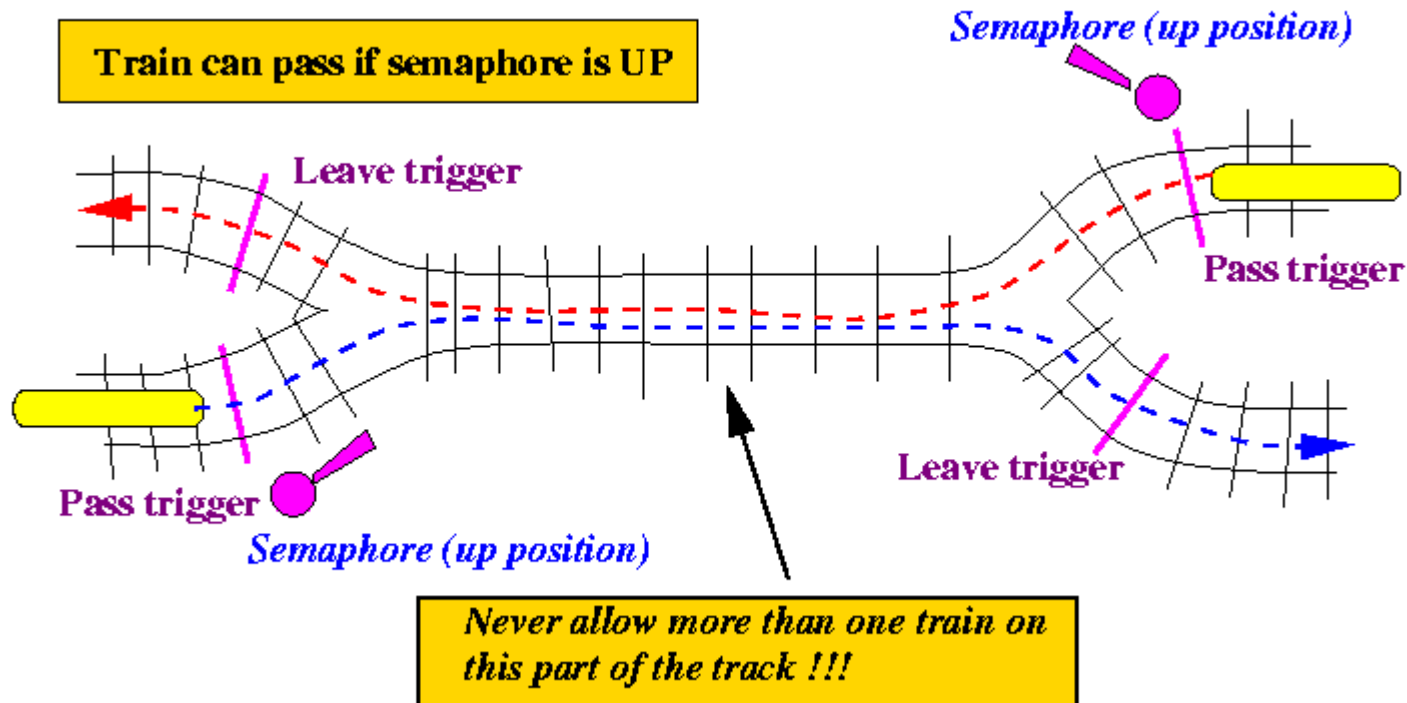
Block if empty

If it is not certain that only a single thread needs to be awakened, it is safer to call **notifyAll()** than **notify()** to make sure that threads are not kept waiting unnecessarily

Exercise

- For the car park scenario, only the code for the **CarParkControl monitor** is provided in the lecture
- Your task is to **implement 2 Java classes**
 - one for *arrivals*
 - one for *departures*(either by extending **Thread** or by implementing the **Runnable** interface)
- Use **multiple threads** to simulate cars **arriving at** or **leaving** the car park
- Print **console messages** to show:
 - car arrivals and departures
 - the current number of free parking spaces
- Introduce **random delays** to simulate arrival and departure times
- **Optional challenge:** go beyond the basics! Add a simple animation or visualisation of the car park
- **Share your solution with us and the other students during the lab session!**

Semaphores: train analogy



- A train entering a “critical section” must pass a “**semaphore**” (signalling) device
- If the semaphore is up, the train can pass, but as soon as the train crosses the semaphore (“pass trigger”), it triggers **all semaphore signals** to go down
- The semaphore signals will only be raised (to “up”) after the train leaves the critical section (triggers the “leave trigger”)

Semaphores

- Semaphores are a **low-level, primitive construct** widely used for dealing with inter-process synchronisation in operating systems
- Introduced by Dijkstra in 1968 – one of the first mechanisms!
- A semaphore **s** is an integer variable that can take only non-negative values.
- **s**: how many units of a particular resource are available, coupled with operations
 - to adjust that record safely (to avoid race conditions) as units are acquired or become free
 - and, if necessary, wait until a unit of the resource becomes available.
- **Counting semaphores**: arbitrary number of resources (> 1)
- **Binary semaphores**: restricted to 0 and 1 (locked/unlocked)

Semaphores

- The only operations permitted on s are **up(s)** and **down(s)**
 - originally called P() and V() from Dutch words *Passeren* = “to pass” and *Vrijgeven* = “to release”
- Blocked processes are held in a FIFO queue

```
down( $s$ ): if  $s > 0$  then  
            decrement  $s$   
            else block execution of the calling process
```

```
up( $s$ ):   if processes blocked on  $s$  then  
            awaken one of them  
            else increment  $s$ 
```

Library analogy (counting semaphore)

- A library has 10 identical study rooms, to be used by one student at a time
 - Students must request a room from the front desk
 - If no rooms are free, students wait at the desk until someone relinquishes a room
 - When a student has finished using a room, the student must return to the desk and indicate that one room has become free
- The clerk at the front desk knows only the number of free rooms available **s**, which they only know correctly if all of the students actually use their room while they've signed up for them and return them when they're done
 - When a student requests a room, the clerk decreases this number
 - When a student releases a room, the clerk increases this number
 - The room can be used for as long as desired, and so it is not possible to book rooms ahead of time

[https://en.wikipedia.org/wiki/Semaphore_\(programming\)](https://en.wikipedia.org/wiki/Semaphore_(programming))

Library analogy (counting semaphore)

- The value of the semaphore in this scenario is initially 10, with all rooms empty
- When a student requests a room, they are granted access, and the value of the semaphore is changed to 9
- After the next student comes, it drops to 8, then 7 and so on
- If someone requests a room and the current value of the semaphore is 0, they are forced to wait until a room is freed (when **S** is increased from 0)
- If one of the rooms was released, but there are several students waiting, then any method can be used to select the one who will occupy the room (like FIFO or flipping a coin)
- Of course, a student needs to inform the clerk about releasing their room only after really leaving it, otherwise, there can be an awkward situation when such student is in the process of leaving the room (they are packing their textbooks, etc.) and another student enters the room before they leave it

[https://en.wikipedia.org/wiki/Semaphore_\(programming\)](https://en.wikipedia.org/wiki/Semaphore_(programming))

Semaphores in Java

- Semaphores are passive objects, therefore implemented as **monitors**
- In practice, semaphores are a low-level mechanism often used for implementing the higher-level monitor construct
- Java provides general counting semaphores (since Java 5)

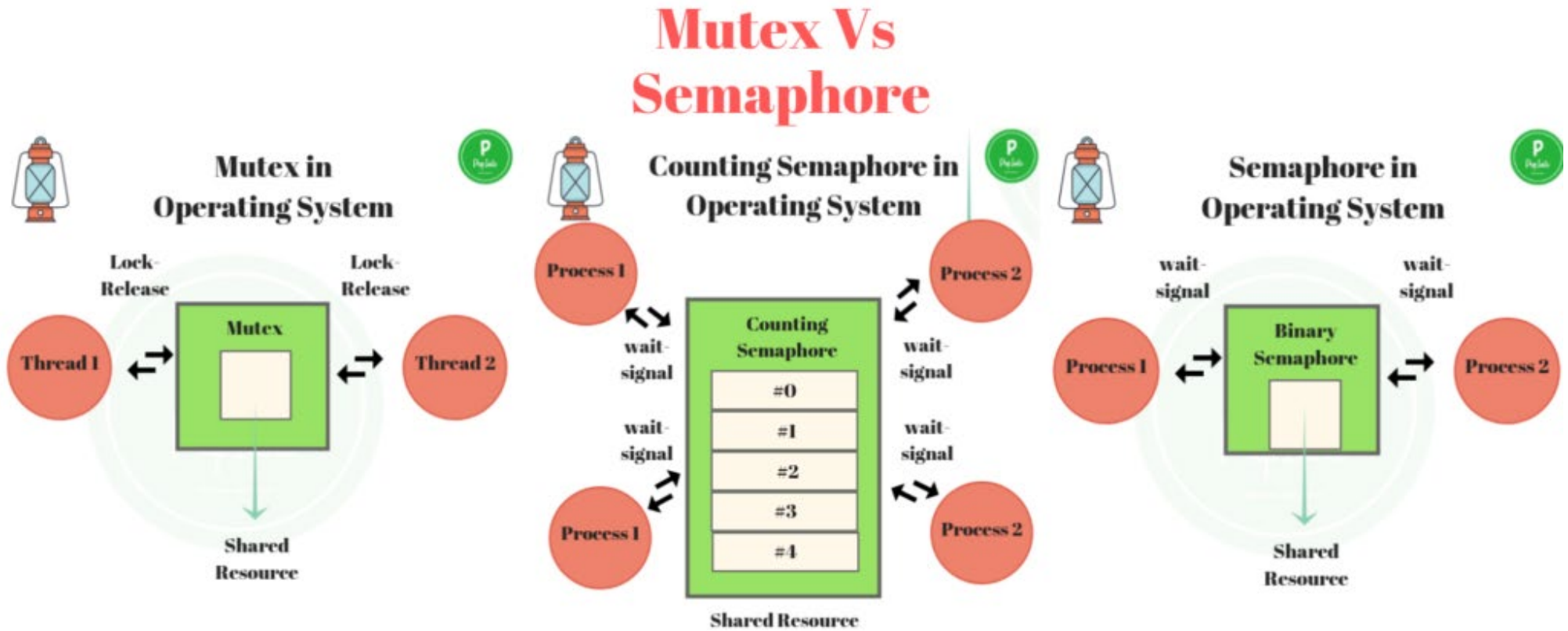
```
public class Semaphore {  
    private int value;  
  
    public Semaphore(int initial)  
        {value = initial;}  
  
    synchronized public void up() {  
        ++value;  
        notifyAll();  
    }  
  
    synchronized public void down()  
        throws InterruptedException {  
        while (value== 0) wait();  
        --value;  
    }  
}
```


Semaphores in Java

Since Java 5, there is a Semaphore class in the package **java.util.concurrent**:

- It supports **down** and **up** operations (called **acquire()** and **release()** in the Java implementation)
- the constructor optionally accepts a **fairness** parameter
 - if this is **false**, the implementation makes no guarantees about the order in which threads are awoken following a **release()**
 - if this is **true**, the semaphore guarantees that threads invoking any of the acquire methods are processed first-in-first-out (FIFO)
- Java implementation of semaphores is based on the higher-level concurrency constructs called... monitors 😊

Mutex vs Semaphore



<https://prepinsta.com/operating-systems/mutex-vs-semaphore/>

Acknowledgments (and resources for further reading)

- Magee & Kramer (2006). Concurrency: State Models and Java Programs (2nd edition), John Wiley. Online slides
<https://www.doc.ic.ac.uk/~jnm/book/>
- Brian Logan. University of Nottingham. Lecture slides
<http://www.cs.nott.ac.uk/~pszbsl/G52CON/>